

**Trabalho (versão 2) - Valor: 10%**

O objetivo deste trabalho é compreender como o ciclo de instrução da Máquina de von Neumann. Para isso, cada **dupla de alunos** deverá implementar um “simulador” de uma CPU com o conjunto de instruções (ISA – *Instruction Set Architecture*), o conjunto de registradores arquiteturais e o formato de instrução listados abaixo. Esse simulador deve mostrar o conteúdo dos registradores no fim de cada ciclo de máquina, quando haverá uma pausa até apertar uma tecla para iniciar o próximo ciclo.

O prazo de entrega é 19/05/2026, domingo, às 23:59 via Moodle. Como de praxe, **cópia ou semelhança nas implementações serão punidas com nota zero** para todas as duplas em que isso for detectado. Passar o seu código para o colega é antiético, antiprofissional e está atrapalhando no desenvolvimento dele. Além disso, corre-se o risco da punição e, no fim do semestre, todo ponto faz a diferença entre ser aprovado ou não. Ainda, **duplas que não tiverem tirado nenhuma dúvida comigo até a entrega do trabalho terão a nota zero atribuída à atividade**. Isso é para evitar alunos que simplesmente apareçam com o trabalho pronto sem nunca ter tirado dúvidas.

**A nota é composta por:**

- **CPU implementada corretamente: 50%**
- **Programa sorteado implementado corretamente: 30%**
- **Apresentação da CPU executando o programa sorteado: 20% (data ainda a ser definida)**

**Os entregáveis são:**

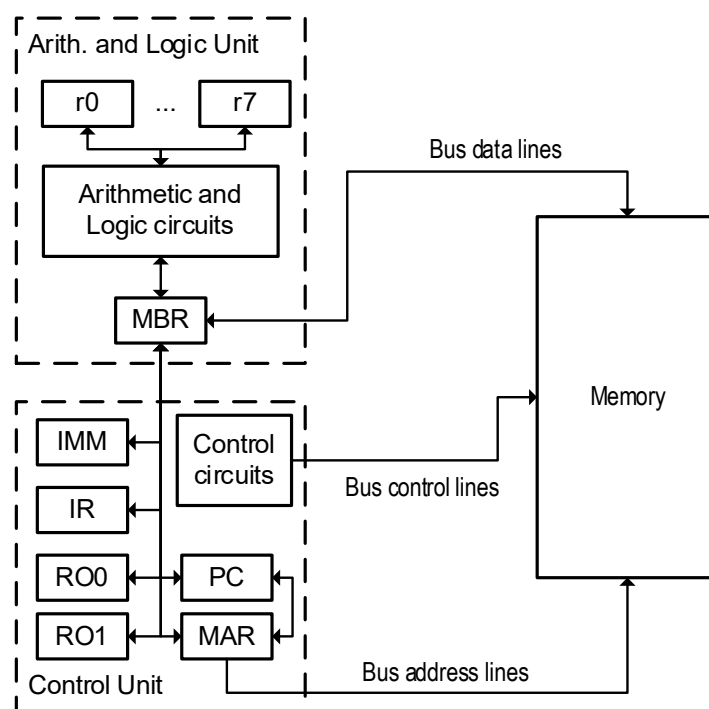
1. **Todos os códigos-fonte necessários para a compilação;**
2. **Instruções de uso do trabalho;**
3. **O trabalho compilado;**
4. **Instruções de compilação, considerando que ele será compilado usando o GCC no Ubuntu;**
5. **O arquivo .txt contendo o programa em assembly sorteado para cada dupla, assim como instruções de uso do programa.**

Ele deve ser implementado em linguagem C. Não é C++ e, sim, C. Portanto, sem orientação a objeto, pois só complica desnecessariamente o desenvolvimento deste trabalho em específico. Para facilitar, **a memória deve ser um vetor de palavras de oito bits** de 256 posições, o que, consequentemente, totalizará essa mesma quantidade em bytes. Em outras palavras, `unsigned char memoria[256]`. Como dica, implemente os registradores como explicado abaixo. **Não serão aceitas** implementações onde os registradores sejam um vetor de caracteres ou a memória, uma matriz de caracteres. O programa deve ser colocado na memória na forma binária respeitando os formatos das palavras de instrução abaixo. Outra característica dessa memória é que ela usa um barramento com oito linhas de dados – ou seja, um barramento com oito bits de largura. Sendo assim, **todas as transferências entre MBR e a memória devem ser de byte em byte**.

O simulador **deve** possuir uma maneira de ler um arquivo texto para carregar a memória com instruções e dados como apresentado na última página. É importante ressaltar, porém, que o simulador não precisa ser “bonito”, mas precisa ser fácil de usar.

Além disso, a CPU a ser implementada processa apenas números inteiros contidos em palavras de 16 bits e, portanto, não há nenhuma operação com ponto-flutuante, BCD ou números inteiros de outros tamanhos. Ademais, não é necessário implementar nenhuma representação de aritmética sinalizada. Embora o formato de instrução permita endereçar até  $2^{16} = 65.536$  palavras de oito bits na memória, a memória possui apenas 256 endereços, como apresentado acima, abrangendo os endereços de 0 (0x0) a 255 (0xFF).

De maneira muito simplificada, o diagrama da CPU com os registradores arquiteturais é:



E a função de cada registrador arquitetural é:

1. **MBR** – *Memory Buffer Register* – contém a palavra a ser armazenada na memória. Também é o registrador usado para receber uma palavra lida da memória. Todo o tráfego de e para a memória RAM deve passar pelo MBR. Teoricamente, ele deveria ter o tamanho de 24 bits, mas como não existe variável de 24 bits na linguagem C, ele deve ser implementado como uma variável de 32 bits (`unsigned int mbr`);
2. **MAR** – *Memory Address Register* – especifica o endereço de memória da palavra a ser lida da ou escrita na memória. Todo endereço de memória deve ser indicado nesse registrador antes da execução da instrução. Deve ser implementado como uma variável de 16 bits (`unsigned short int mar`);
3. **IR** – *Instruction Register* – contém o *opcode* da instrução a ser executada. Teoricamente, ele deveria ter o tamanho de cinco bits, que não existe na linguagem C. Por isso, deve ser implementado como uma variável de oito bits (`unsigned char ir`);
4. **RO0** – *Register Operand 0* – contém o endereço do primeiro operando registrador da instrução. Teoricamente, ele deveria ter o tamanho de três bits, que não existe na linguagem C. Por isso, deve ser implementado como uma variável de oito bits (`unsigned char ro0`);
5. **RO1** – *Register Operand 1* – contém o endereço do segundo operando registrador da instrução. Teoricamente, ele deveria ter o tamanho de três bits, que não existe na linguagem C. Por isso, deve ser implementado como uma variável de oito bits (`unsigned char ro1`);
6. **IMM** – *Immediate* – contém o operando imediato da instrução. Deve ser uma variável de 16 bits (`unsigned short int imm`).
7. **PC** – *Program Counter* – contém o endereço da próxima palavra de instrução a ser buscada na memória. Caso não haja nenhum desvio, `halt` ou `nop`, o PC deve ser incrementado em cada ciclo de instrução tendo como referência o tamanho da instrução recém-executada. Deve ser uma variável de 16 bits (`unsigned short int pc`);
8. **E, L e G** – registradores internos que armazenam as *flags* 'equal to', 'lower than' e 'greater than'. Cada uma delas contém um bit indicando se o conteúdo do primeiro operando registrador é, ao ser comparado pela instrução `cmp`, respectivamente 1) igual a, 2) menor do que ou 3) maior do que o conteúdo do segundo operando registrador. Como não há maneira de implementar variáveis de um bit, devem ser implementados como variáveis de oito bits (`unsigned char e`, `unsigned char l`, `unsigned char g`);

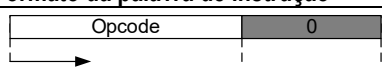
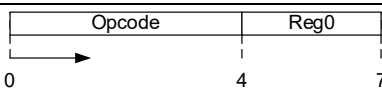
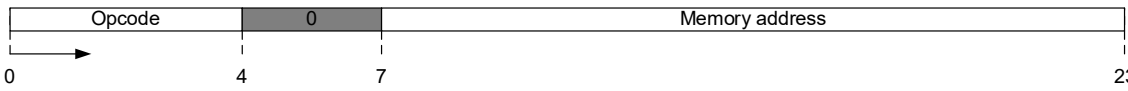
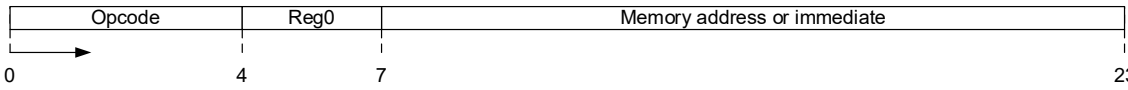
9. **r0 a r7** – registradores de propósito-geral (GPRs) utilizados para manter temporariamente os operandos na ALU. Devem ser implementados como um vetor com posições de 16 bits (unsigned short int reg[8]). Esses registradores são codificados nos campos de instrução **reg0** e **reg1** da seguinte forma:

| Reg. | Sequência de bits |
|------|-------------------|
| r0   | 000               |
| r1   | 001               |
| r2   | 010               |
| r3   | 011               |
| r4   | 100               |
| r5   | 101               |
| r6   | 110               |
| r7   | 111               |

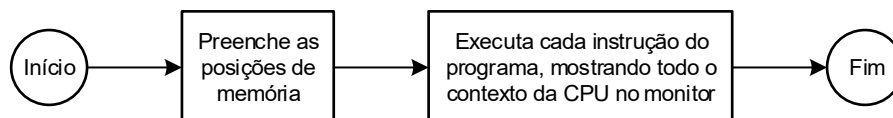
Finalmente, o conjunto de instruções é:

| Mnemônico    | Opcode   | Descrição                                                                                                                                                                                                                                                                                                                                                  |
|--------------|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hlt          | 0b000000 | HALT: o processador não faz nada. Em outras palavras, <u>nenhum</u> registrador tem o seu valor alterado durante a execução de hlt. Deve-se colocar no fim do programa.                                                                                                                                                                                    |
| nop          | 0b000001 | NO OPERATION: O PC é incrementado, mas <u>nenhum</u> outro registrador tem seu valor alterado durante a execução de nop.                                                                                                                                                                                                                                   |
| ldr rX, rY   | 0b000010 | LOAD VIA REGISTER: $rX = *rY$ , considerando que rX possui 16 bits. Em outras palavras, o registrador rX tem que ser carregado com os valores nos endereços de memória rY e (rY + 1).                                                                                                                                                                      |
| str rX, rY   | 0b000011 | STORE VIA REGISTER: $*rY = rX$ , considerando que rX possui 16 bits. Em outras palavras, o valor armazenado no registrador rX tem que ser armazenado nos endereços de memória rY e (rY + 1).                                                                                                                                                               |
| add rX, rY   | 0b001000 | ADD REGISTER: $rX = rX + rY$                                                                                                                                                                                                                                                                                                                               |
| sub rX, rY   | 0b001001 | SUBTRACT REGISTER: $rX = rX - rY$                                                                                                                                                                                                                                                                                                                          |
| mul rX, rY   | 0b001010 | MULTIPLY REGISTER: $rX = rX \times rY$                                                                                                                                                                                                                                                                                                                     |
| div rX, rY   | 0b001011 | DIVIDE REGISTER: $rX = rX \div rY$                                                                                                                                                                                                                                                                                                                         |
| cmp rX, rY   | 0b010000 | COMPARE REGISTER: compara a palavra no registrador rX com a palavra no registrador rY e preenche os registradores internos E, L e G os valores fazendo sequencialmente os seguintes testes:<br>1. Se $rX = rY$ , então $E = 1$ ; senão $E = 0$ ;<br>2. Se $rX < rY$ , então $L = 1$ ; senão $L = 0$ ;<br>3. Se $rX > rY$ , então $G = 1$ ; senão $G = 0$ . |
| movr rX, rY  | 0b010001 | MOVE REGISTER: $rX = rY$                                                                                                                                                                                                                                                                                                                                   |
| and rX, rY   | 0b010010 | LOGICAL-AND ON REGISTER: $rX = rX \& rY$                                                                                                                                                                                                                                                                                                                   |
| or rX, rY    | 0b010011 | LOGICAL-OR ON REGISTER: $rX = rX   rY$                                                                                                                                                                                                                                                                                                                     |
| xor rX, rY   | 0b010100 | LOGICAL-XOR ON REGISTER: $rX = rX \wedge rY$                                                                                                                                                                                                                                                                                                               |
| not rX       | 0b010101 | LOGICAL-NOT ON REGISTER: $rX = !rX$                                                                                                                                                                                                                                                                                                                        |
| je Z         | 0b010110 | JUMP IF EQUAL TO: muda o registrador PC para o endereço de memória Z caso $E = 1$ .                                                                                                                                                                                                                                                                        |
| jne Z        | 0b010111 | JUMP IF NOT EQUAL TO: muda o registrador PC para o endereço de memória Z caso $E = 0$ .                                                                                                                                                                                                                                                                    |
| jl Z         | 0b100000 | JUMP IF LOWER THAN: muda o registrador PC para o endereço de memória Z caso $L = 1$ .                                                                                                                                                                                                                                                                      |
| jle Z        | 0b100001 | JUMP IF LOWER THAN OR EQUAL TO: muda o registrador PC para o endereço de memória Z caso $E = 1$ ou $L = 1$ .                                                                                                                                                                                                                                               |
| jg Z         | 0b100010 | JUMP IF GREATER THAN: muda o registrador PC para o endereço de memória Z caso $G = 1$ .                                                                                                                                                                                                                                                                    |
| jge Z        | 0b100011 | JUMP IF GREATER THAN OR EQUAL TO: muda o registrador PC para o endereço de memória Z caso $E = 1$ ou $G = 1$ .                                                                                                                                                                                                                                             |
| jmp Z        | 0b101000 | JUMP: muda o registrador PC para o endereço de memória Z.                                                                                                                                                                                                                                                                                                  |
| ld rX, Z     | 0b101001 | LOAD: carrega o registrador rX com uma palavra da memória de 16 bits que se inicia no endereço Z, ou seja, $rX = *Z$ . Em outras palavras, o registrador rX tem que ser carregado com os valores nos endereços de memória Z e (Z + 1).                                                                                                                     |
| st rX, Z     | 0b101010 | STORE: armazena uma palavra de 16 bits que começa a partir do endereço de memória Z o conteúdo do registrador rX, ou seja, $*Z = rX$ . Em outras palavras, o valor armazenado no registrador rX tem que ser armazenado nos endereços de memória Z e (Z + 1).                                                                                               |
| movi rX, IMM | 0b101011 | MOVE IMMEDIATE: $rX = IMM$                                                                                                                                                                                                                                                                                                                                 |
| addi rX, IMM | 0b110000 | ADD IMMEDIATE: $rX = rX + IMM$                                                                                                                                                                                                                                                                                                                             |
| subi rX, IMM | 0b110001 | SUBTRACT IMMEDIATE: $rX = rX - IMM$                                                                                                                                                                                                                                                                                                                        |
| muli rX, IMM | 0b110010 | MULTIPLY IMMEDIATE: $rX = rX \times IMM$                                                                                                                                                                                                                                                                                                                   |
| divi rX, IMM | 0b110011 | DIVIDE IMMEDIATE: $rX = rX \div IMM$                                                                                                                                                                                                                                                                                                                       |
| lsh rX, IMM  | 0b111000 | LEFT SHIFT: desloca a palavra no registrador rX em IMM bits à esquerda.                                                                                                                                                                                                                                                                                    |
| rsh rX, IMM  | 0b111001 | RIGHT SHIFT: desloca a palavra no registrador rX em IMM bits à direita.                                                                                                                                                                                                                                                                                    |

As instruções são codificadas da seguinte forma:

| Mnemônico    | Formato da palavra de instrução                                                      |
|--------------|--------------------------------------------------------------------------------------|
| hlt          |     |
| nop          |     |
| ldr rX, rY   |    |
| str rX, rY   |                                                                                      |
| add rX, rY   |                                                                                      |
| sub rX, rY   |                                                                                      |
| mul rX, rY   |                                                                                      |
| div rX, rY   |                                                                                      |
| cmp rX, rY   |                                                                                      |
| movrr rX, rY |                                                                                      |
| and rX, rY   |                                                                                      |
| or rX, rY    |                                                                                      |
| xor rX, rY   |                                                                                      |
| not rX       |     |
| je Z         |    |
| jne Z        |                                                                                      |
| j1 Z         |                                                                                      |
| jle Z        |                                                                                      |
| jg Z         |                                                                                      |
| jge Z        |                                                                                      |
| jmp Z        |                                                                                      |
| ld rX, Z     |  |
| st rX, Z     |                                                                                      |
| movi rX, IMM |                                                                                      |
| addi rX, IMM |                                                                                      |
| subi rX, IMM |                                                                                      |
| muli rX, IMM |                                                                                      |
| divi rX, IMM |                                                                                      |
| lsh rX, IMM  |                                                                                      |
| rsh rX, IMM  |                                                                                      |

Observe que a CPU simulada deverá ser capaz de executar **qualquer** programa que for adicionado à memória. O programa deve ser colocado na memória na forma binária respeitando os formatos das palavras de instrução ilustradas na tabela acima. Dessa forma, o funcionamento do programa deve seguir a seguinte sequência:



Para a primeira etapa – carregar a memória – o simulador deve ler um arquivo texto como apresentado na última página. Essa etapa deve continuar enquanto o programa não encontrar a linha contendo 'hlt' (sem os apóstrofes). A conversão preencherá a memória em uma maneira semelhante à observada abaixo.

| End. mem. | Conteúdo    | Conteúdo codificado em binário | Conteúdo codificado em hexadecimal |
|-----------|-------------|--------------------------------|------------------------------------|
| 0x0       | ld r0, 1E   | 1010 1000 0000 0000 0001 1110  | A8 00 1E                           |
| 0x3       | ld r1, 20   | 1010 1001 0000 0000 0010 0000  | A9 00 20                           |
| 0x6       | add r0, r1  | 0010 0000 0010 0000            | 20 20                              |
| 0x8       | addi r0, 14 | 1100 0000 0000 0000 0001 0100  | C0 00 14                           |
| 0xB       | st r0, 22   | 1011 0000 0000 0000 0010 0010  | B0 00 22                           |
| 0xE       | hlt         | 0000 0000                      | 00 00                              |
| ...       | ...         | ...                            | ...                                |
| 0x1E      | 15          | 0000 0000 0000 1111            | 00 0F                              |
| 0x20      | 8           | 0000 0000 0000 1000            | 00 08                              |

Você pode observar na última coluna da tabela acima que os valores estão agrupados em pares de valores hexadecimais, onde cada um deles é um byte (ou o equivalente a uma posição de memória). Dessa forma, o mapa de memória fica assim (os endereços de memória e os seus conteúdos estão em hexadecimal):

| Endereço | Conteúdo | Endereço | Conteúdo | Endereço | Conteúdo | Endereço | Conteúdo |
|----------|----------|----------|----------|----------|----------|----------|----------|
| 0        | A8       | A        | 14       | 14       | ...      | 1E       | 00       |
| 1        | 00       | B        | B0       | 15       | ...      | 1F       | 0F       |
| 2        | 1E       | C        | 00       | 16       | ...      | 20       | 00       |
| 3        | A9       | D        | 22       | 17       | ...      | 21       | 08       |
| 4        | 00       | E        | 00       | 18       | ...      | 22       | ...      |
| 5        | 20       | F        | 00       | 19       | ...      | 23       | ...      |
| 6        | 20       | 10       | ...      | 1A       | ...      | 24       | ...      |
| 7        | 20       | 11       | ...      | 1B       | ...      | 25       | ...      |
| 8        | C0       | 12       | ...      | 1C       | ...      | 26       | ...      |
| 9        | 00       | 13       | ...      | 1D       | ...      | 27       | ...      |

O que estará na memória é o que está nessa tabela.

Na segunda etapa – que é exibir o funcionamento da CPU executando o programa – o trabalho deve exibir algo assim (considerando que todos os conteúdos de registradores e da memória, incluindo os seus endereços, estão em hexadecimal):

```

CPU:
R0:  FFFF  R1:  FFFF  R2:  FFFF  R3:  FFFF
R4:  FFFF  R5:  FFFF  R6:  FFFF  R7:  FFFF
MBR:  FFFFFFFF  MAR:  FFFF  IMM:  FFFF  PC:  FFFF
IR:   FF      RO0:  F      RO1:  F
E:    F      L:    F      G:    F

```

Memória:

```

00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
00 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
10 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
20 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
30 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
40 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
50 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
60 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
70 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
80 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
90 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
A0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
B0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
C0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
D0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
E0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
F0 FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF

```

Pressione uma tecla para iniciar o próximo ciclo de máquina ou aperte CTRL+C para finalizar a execução do trabalho.

Deve-se exibir o conteúdo dos registradores e da memória ao fim de cada ciclo de máquina, com o usuário devendo pressionar uma tecla para iniciar a execução do próximo ciclo.

Para ajudar no desenvolvimento do trabalho, teste-o com os dois programas abaixo, considerando que eles já estão no formato esperado de arquivo esperado de arquivo a ser lido pelo trabalho. Todos os dados, endereços e imediatos estão representados em hexadecimal. Observe que as linhas que estão no formato endereço;instrução/dado;palavra de instrução ou palavra de dado. Em todo caso, o endereço inicial de memória que aquela instrução ou dado ocupará é dado em hexadecimal, assim como os valores numéricos nas instruções. Além disso, observe que as instruções possuem tamanho variável – de um a três bytes. Os dados sempre ocupam dois bytes.

|                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $A = 32 + 3 \times \frac{4}{5 - 3}$                                                                                                                                                                         | $A = \sum_{1}^{10} \frac{10}{5} + 3 \times (2 - 1)$                                                                                                                                                                                                                                                                                                    |
| <pre>0;i;ld r0, 96 3;i;ld r1, 98 6;i;sub r0, r1 8;i;ld r1, 94 b;i;div r1, r0 d;i;ld r2, 92 10;i;mul r2, r1 12;i;ld r1, 90 15;i;add r1, r2 17;i;st r1, 8e 1a;i;hlt 90;d;20 92;d;3 94;d;4 96;d;5 98;d;3</pre> | <pre>0;i;ld r0, 90 3;i;ld r1, 92 6;i;div r0, r1 8;i;ld r1, 96 b;i;ld r2, 98 e;i;sub r1, r2 10;i;ld r2, 94 13;i;mul r1, r2 15;i;add r0, r1 17;i;ld r1, 8a 1a;i;add r0, r1 1c;i;st r0, 8a 1f;i;ld r0, 8c 22;i;ld r1, 8e 25;i;addi r0, 1 28;i;st r0, 8c 2b;i;cmp r0, r1 2d;i;jle 0 30;i;hlt 8a;d;0 8c;d;1 8e;d;a 90;d;a 92;d;5 94;d;3 96;d;2 98;d;1</pre> |

Programas que devem ser feitos em *assembly* (sorteado para cada dupla):

1. Cálculo iterativo da sequência de Fibonacci com tamanho definido em um valor de 16 bits armazenado em 0xFC e 0xFD. O resultado de 16 bits deve ser armazenado em 0xFE e 0xFF.
2. Produto interno entre dois vetores de dez posições de 16 bits cada. O resultado é um valor de 16 bits deve ser armazenado em 0xFE e 0xFF. O vetor **A** deve ser armazenado nos endereços 0xD6 a 0xE9 e o vetor **B** nos endereços 0xEA a 0xFD.
3. Soma de duas matrizes  $3 \times 3$ , onde cada valor possui 16 bits. A matriz **A** deve ser armazenada nos endereços 0xCA a 0xDB, enquanto a matriz **B** deve ser armazenada nos endereços 0xDC a 0xED. O resultado da soma das matrizes deve ser armazenado nos endereços 0xEE a 0xFF.
4. Um programa que calcula a conta de energia de um consumidor tendo como entrada o consumo dado em KWh. Ele pode ser residencial (0x1), comercial (0x2) ou industrial (0x3). O valor da tarifa do KWh consumido para cada um dos tipos de consumidor é 0x10, 0x20 e 0x30, respectivamente. O consumo em KWh é um valor de 16 bits e deve ser armazenado em 0xFE o 0xFF enquanto o valor da conta de energia – também um valor de 16 bits – deve ficar em 0xFC e 0xFD.
5. Um programa que conta em um vetor de 20 posições de 16 bits cada quantos valores são maiores do que 0x20. O vetor deve estar armazenado entre 0xD8 e 0xFD. O resultado final deve estar armazenado em um valor de 16 bits nos endereços 0xFE e 0xFF.
6. Um programa que armazena dez cópias de um valor de 16 bits em outras posições de memória. O valor a ser copiado ocupa os endereços 0xFE e 0xFF. O ponteiro que aponta para os endereços onde vai ser armazenada uma cópia fica nos endereços 0xFC e 0xFD. Para cada laço do loop de iteração, deve-se incrementar o ponteiro em 0x2.
7. Um programa que altera um vetor de 20 posições de 16 bits cada. O vetor deve estar armazenado entre 0xD8 e 0xFD. Caso o elemento do vetor seja menor do que 0x10, o novo valor deve ser 0xFF. Caso o elemento do vetor seja maior do que 0x15, o novo valor passa a ser 0xDD.